Guía de Estudio
Patrones de Diseño

Diseño de Software — Grado en Ingeniería Informática
Universidad de Santiago de Compostela
Curso 2025/2026

Autoría

Iago Leis Fernández
iago.leis@rai.usc.es

Nuria Guerra Casal
nuria.guerra.casal@rai.usc.es

Temas cubiertos

T8. Patrones de Creación | T9. Patrones Estructurales | T10. Patrones de Comportamiento

Aviso importante: este material es de apoyo al estudio y su contenido **no es 100% fiable**; puede contener errores, omisiones o interpretaciones imprecisas. Se prohíbe expresamente su **venta**, distribución con ánimo de lucro y su **apropiación** o presentación como obra propia.

Documento generado para preparación del ejercicio 3.

Índice

1 Introducción a los Patrones de Diseño **2**

2 Patrones de Creación (T8) **3**
2.1 Abstract Factory (Fábrica Abstracta) . . . 3
2.1.1 Motivación y ejemplo . . . 3
2.2 Factory Method (Método de Fabricación) . . . 4
2.2.1 Motivación y ejemplo . . . 4

3 Patrones Estructurales (T9) **5**
3.1 Adapter (Adaptador) . . . 5
3.1.1 Motivación y ejemplo . . . 5
3.2 Bridge (Puente) . . . 5
3.2.1 Motivación y ejemplo . . . 6
3.3 Composite . . . 6
3.3.1 Motivación y ejemplo . . . 6
3.4 Decorator (Decorador) . . . 7
3.4.1 Motivación y ejemplo . . . 7
3.5 Proxy (Apoderado) . . . 8
3.5.1 Tipos de Proxy . . . 8
3.5.2 Ejemplo . . . 8

4 Patrones de Comportamiento (T10) **10**
4.1 Observer (Observador) . . . 10
4.1.1 Motivación y ejemplo . . . 10
4.2 State (Estado) . . . 11
4.2.1 Motivación y ejemplo . . . 11
4.3 Strategy (Estrategia) . . . 12
4.3.1 Motivación y ejemplo . . . 12
4.4 Template Method (Método Plantilla) . . . 12
4.4.1 Motivación y ejemplo . . . 12
4.4.2 Tipos de operaciones invocadas por el método plantilla . . . 13

5 Tablas Resumen para el Examen **14**
5.1 Resumen general de los patrones . . . 14
5.2 Comparación de patrones similares . . . 14
5.3 Composición y patrones que se relacionan . . . 15
5.4 Tabla de participantes rápida . . . 16

Introducción a los Patrones de Diseño

Los **patrones de diseño** son soluciones reutilizables a problemas recurrentes en el diseño de software orientado a objetos. Fueron popularizados por el libro de Gamma et al. (el "Gang of Four" o GoF). Proporcionan un **vocabulario común** que eleva la abstracción y reduce la complejidad del sistema.

Se clasifican en tres categorías según su propósito:

Categoría	Descripción
Creación	Abstraen el proceso de creación de objetos. Ocultan cómo se crean las instancias. Permiten programar para interfaces, no implementaciones.
Estructurales	Tratan la composición de clases y objetos para formar estructuras más grandes.
Comportamiento	Definen cómo interactúan y se comunican los objetos.

⋆ Recuerda para el examen

En el examen suele pedirse: **(1) identificar el patrón** en un diagrama o descripción, **(2) nombrar sus participantes**, **(3) explicar cuándo usarlo**, y **(4) compararlo con otros patrones similares**. Presta especial atención a los diagramas de estructura UML.

Patrones de Creación (T8)

Los patrones de creación permiten abstraerse de la creación de objetos, acotando el conocimiento sobre las clases concretas utilizadas. Ocultan cómo se crean y enlazan las instancias, de manera que el resto del sistema sólo conoce los objetos a través de sus interfaces.

2.1. Abstract Factory (Fábrica Abstracta)

Abstract Factory — Idea clave

Proporciona una **interfaz para crear familias de objetos relacionados** sin especificar sus clases concretas. El cliente solo conoce las interfaces de los productos, no sus implementaciones.

Motivación y ejemplo

Imagina un toolkit de interfaz gráfica que debe funcionar en KDE y en GNOME. Se definen interfaces `IVentana` e `IBoton` que representan los componentes. Las clases `VentanaKDE`, `VentanaGNOME`, `BotonKDE` y `BotonGNOME` implementan dichas interfaces. La **fábrica abstracta** `IFabricaControles` declara métodos `crearVentana()` y `crearBoton()`. Las fábricas concretas `FabricaKDE` y `FabricaGNOME` implementan esta interfaz creando la familia correspondiente. El cliente solo depende de `IFabricaControles` y de las interfaces de producto: no sabe qué familia está usando.

¿Cuándo usarlo?

- Para configurar un sistema escogiendo **una sola familia de productos** entre varias.
- Cuando cada familia está diseñada para el uso conjunto de sus productos (coherencia).
- Para proporcionar bibliotecas de productos de los que sólo se revelan sus interfaces.

Participantes

- **FabricaAbstracta**: declara la interfaz para crear productos abstractos.
- **FabricaConcreta**: implementa los métodos de creación para una familia específica.
- **ProductoAbstracto**: declara la interfaz de un tipo de producto.
- **ProductoConcreto**: implementa la interfaz del producto para una familia concreta.
- **Cliente**: usa solo las interfaces de fábrica y producto.

Ventajas e inconvenientes

Ventajas:

- Desacopla clientes de implementaciones concretas.
- Facilita el cambio de familias de productos (basta cambiar la fábrica).
- Garantiza consistencia entre productos de la misma familia.

Inconvenientes:

- Difícil añadir nuevos *tipos* de productos (habría que modificar la interfaz de la fábrica y todas sus implementaciones).

2.2. Factory Method (Método de Fabricación)

Factory Method — Idea clave

Define una **interfaz para crear un objeto**, pero deja que las **subclases decidan qué clase instanciar**. Permite que una clase difiera la instanciación a sus subclases.

Motivación y ejemplo

En una aplicación de documentos, la clase abstracta `Aplicacion` contiene el método plantilla `abrirDoc()` que llama al método de fabricación `crearDoc()`. La subclase `MiAplicacion` redefine `crearDoc()` para devolver una instancia de `MiDocumento`. Así, `Aplicacion` controla el flujo general sin conocer la clase concreta del documento.

¿Cuándo usarlo?

- Cuando una clase no puede prever la clase de objetos que debe crear.
- Cuando se quiere que las subclases especifiquen los objetos que crean.
- Para proporcionar enganches a subclases con versiones extendidas de un objeto.

Participantes

- **Producto**: define la interfaz de los objetos creados.
- **ProductoConcreto**: implementa la interfaz Producto.
- **Creador**: declara el método de fabricación; puede implementar una versión por defecto.
- **CreadorConcreto**: redefine el método de fabricación para devolver un ProductoConcreto.

Ventajas e inconvenientes

Ventajas:

- Los clientes solo tratan con la interfaz `Producto`.
- Permite conectar jerarquías paralelas de clases.
- Dota a subclases de enganches para versiones extendidas.

Inconvenientes:

- Los clientes pueden necesitar heredar de `Creador` solo para crear un ProductoConcreto.

⋆ Recuerda para el examen

Abstract Factory vs. Factory Method:

- **Factory Method**: usa herencia; la subclase decide qué crear. Más simple, un producto.
- **Abstract Factory**: usa composición/delegación; un objeto fábrica crea *familias* de productos. Más flexible y complejo.
- Abstract Factory suele implementarse internamente con varios Factory Methods.

Patrones Estructurales (T9)

Los patrones estructurales tratan la composición de clases y objetos para construir estructuras más grandes y flexibles. Emplean herencia o composición para definir nuevas funcionalidades.

3.1. Adapter (Adaptador)

Adapter — Idea clave

Convierte la **interfaz de una clase en otra** que esperan sus clientes. Permite cooperar a clases con interfaces incompatibles (actúa como un "enchufe" entre dos sistemas).

Motivación y ejemplo

El editor de dibujo `EditorDibujo` trabaja con objetos `IForma` que tienen `cajaLimitrofe()` y `crearManipulador()`. Existe una clase de terceros `VistaTexto` con el método `obtenerExtension()` pero que no implementa `IForma`. La clase adaptador `FormaTexto` implementa `IForma` y delega internamente a `VistaTexto`, haciendo la adaptación transparente al cliente.

¿Cuándo usarlo?

- Cuando se quiere usar una clase existente cuya interfaz no concuerda con la necesaria.
- Para crear una clase reutilizable que coopere con clases no previstas.
- Cuando es necesario usar varias subclases existentes sin heredar de cada una.

Participantes

- **Objetivo**: define la interfaz específica del dominio que el Cliente espera.
- **Cliente**: colabora con objetos que se ajustan a la interfaz Objetivo.
- **Adaptado**: la clase existente con interfaz incompatible que se desea reutilizar.
- **Adaptador**: implementa la interfaz Objetivo y delega al Adaptado.

Ventajas e inconvenientes

Ventajas:

- Un mismo Adaptador puede funcionar con muchos Adaptados.
- Permite añadir funcionalidad al adaptar.

Inconvenientes:

- Es más difícil redefinir el comportamiento del Adaptado.

3.2. Bridge (Puente)

Bridge — Idea clave

Separa una abstracción de su implementación de forma que ambas puedan variar de manera independiente. Evita la explosión de subclases al introducir dos jerarquías separadas.

Motivación y ejemplo

Si una clase `Ventana` necesita implementaciones para KDE y GNOME, y además existen subtipos como `VentanaIcono` y `VentanaFlotante`, el enfoque de herencia simple produciría 4 combinaciones (`VentanaIconoKDE`, `VentanaIconoGNOME`, etc.). Con Bridge se separa la jerarquía de abstracciones (`Ventana`, `VentanaIcono`, `VentanaFlotante`) de la jerarquía de implementaciones (`VentanaImpKDE`, `VentanaImpGNOME`), conectadas mediante una referencia `-imp`.

¿Cuándo usarlo?

- Para evitar un vínculo permanente entre abstracción e implementación (selección en tiempo de ejecución).
- Cuando abstracción e implementación deban ser ampliables mediante herencia.
- Para impedir que los cambios en la implementación afecten a los clientes.
- Para compartir una implementación entre varios objetos ocultándola.

Participantes

- **Abstraccion**: define la interfaz de la abstracción y mantiene una referencia al Implementador.
- **AbstraccionRefinada**: extiende la interfaz definida por Abstraccion.
- **Implementador**: define la interfaz para las clases de implementación (operaciones de más bajo nivel).
- **ImplementadorConcreto**: implementa la interfaz Implementador.

Ventajas e inconvenientes

Ventajas:

- Desacopla interfaz e implementación; la implementación se puede cambiar en tiempo de ejecución.
- Elimina dependencias de compilación.
- Permite desarrollar abstracción e implementación de forma independiente.
- Oculta detalles de implementación a los clientes.

⋆ Recuerda para el examen

Adapter vs. Bridge: Adapter resuelve incompatibilidades *existentes* (se aplica después del diseño). Bridge planifica la separación *desde el principio* para que abstracción e implementación evolucionen independientemente. Bridge *hace que las cosas funcionen antes de ser diseñadas*; Adapter *lo hace después*.

3.3. Composite

Composite — Idea clave

Organiza objetos en **estructuras de árbol** para representar jerarquías parte-todo. Permite tratar de manera **uniforme** a objetos individuales (hojas) y a composiciones de objetos (compuestos).

Motivación y ejemplo

En un editor gráfico, `IGraphico` es la interfaz común con operaciones `dibujar()`, `trasladar()`, etc. Las clases `Linea`, `Rectangulo` y `Texto` son hojas que la implementan. La clase `Dibujo` es

un compuesto que también implementa `IGraphico` y contiene una colección de `IGraphico`; su método `dibujar()` itera sobre todos sus hijos y llama a `g.dibujar()`. El cliente puede tratar un `Dibujo` exactamente igual que una `Linea`.

¿Cuándo usarlo?

- Para representar jerarquías de objetos parte-todo.
- Para que los clientes puedan ignorar la diferencia entre composiciones e individuos.

Participantes

- **Componente**: declara la interfaz para los objetos de la jerarquía; implementa comportamiento predeterminado.
- **Hoja**: representa los objetos primitivos sin hijos; implementa el comportamiento básico.
- **Compuesto**: almacena hijos, implementa operaciones de gestión de hijos (`añadir`, `eliminar`, `obtenerHijo`), y delega operaciones a sus hijos.
- **Cliente**: manipula objetos a través de la interfaz Componente.

Ventajas e inconvenientes

Ventajas:

- Permite la agregación recursiva.
- Uniformiza el acceso: el cliente no distingue hojas de compuestos.
- Facilita la adición de nuevos tipos de componentes.

Inconvenientes:

- Dificulta restringir qué componentes pueden pertenecer a un grupo.

3.4. Decorator (Decorador)

Decorator — Idea clave

Asigna responsabilidades adicionales a un objeto de manera dinámica, sin modificar su clase. Ofrece una alternativa flexible a la herencia para extender funcionalidad.

Motivación y ejemplo

Una clase `VistaTexto` implementa `IComponenteVisual`. Se desea añadir dinámicamente scroll y borde. Se crea una clase abstracta `Decorador` que también implementa `IComponenteVisual` y contiene una referencia a un `IComponenteVisual`. Las subclases concretas `DecoradorDesplazamiento` y `DecoradorBorde` añaden su funcionalidad antes/después de delegar al componente contenido. Se pueden encadenar decoradores: `new DecoradorBorde(new DecoradorDesplazamiento(new VistaTexto()))`.

¿Cuándo usarlo?

- Para añadir o quitar responsabilidades a objetos individuales de forma dinámica y transparente.
- Cuando la extensión mediante herencia no es viable (clases `final` o explosión de subclases).

Participantes

- **Componente**: interfaz para los objetos que pueden ser decorados.
- **ComponenteConcreto**: el objeto real al que se añaden responsabilidades.
- **Decorador**: mantiene una referencia a un Componente y se ajusta a su interfaz.
- **DecoradorConcreto**: añade las responsabilidades concretas al componente.

Ventajas e inconvenientes

Ventajas:

- Mayor flexibilidad que la herencia (que es estática).
- Evita clases con demasiadas responsabilidades en la parte superior de la jerarquía.

Inconvenientes:

- Da lugar a muchos objetos pequeños y similares, lo que dificulta la depuración.

3.5. Proxy (Apoderado)

Proxy — Idea clave

Proporciona un **sustituto o representante de otro objeto** para controlar el acceso a él. La interfaz del Proxy es **idéntica** a la del objeto representado.

Tipos de Proxy

Tipo	Descripción
Proxy remoto	Representa localmente un objeto en otro espacio de direcciones (red).
Proxy virtual	Crea objetos de alto coste sólo cuando son realmente necesarios (lazy loading).
Proxy de protección	Controla el acceso al objeto real según permisos.
Referencia inteligente	Sustituye a un puntero simple y realiza operaciones adicionales (conteo de referencias, logging).

Ejemplo

En un editor de documentos, las imágenes son costosas de cargar. `ProxyImagen` implementa `IGrafico` igual que `Imagen`. Cuando el cliente llama a `dibujar()`, el proxy verifica si la imagen real ya está cargada; si no, la instancia en ese momento (lazy loading). Para `obtenerExtension()` puede responder directamente sin cargar la imagen completa.

Participantes

- **Sujeto**: define la interfaz común para SujetoReal y Proxy.
- **SujetoReal**: el objeto real que el Proxy representa.
- **Proxy**: controla el acceso al SujetoReal y puede gestionar su creación/destrucción.

⋆ Recuerda para el examen

Decorator vs. Proxy vs. Adapter:

- **Adapter**: cambia la *interfaz* de un objeto.
- **Proxy**: misma interfaz que el original; controla el *acceso*.
- **Decorator**: misma interfaz que el original; añade *responsabilidades* dinámicamente.
- Proxy y Decorator tienen implementación similar pero propósito distinto.

Patrones de Comportamiento (T10)

Los patrones de comportamiento se ocupan de los algoritmos y la asignación de responsabilidades entre objetos. Describen no solo los patrones de objetos o clases, sino también los patrones de comunicación entre ellos.

4.1. Observer (Observador)

Observer — Idea clave

Define una **dependencia uno-a-muchos** entre objetos, de manera que cuando un objeto (sujeto) cambia su estado, todos sus dependientes (observadores) son **notificados y actualizados automáticamente**.

Motivación y ejemplo

En una hoja de cálculo, los mismos datos pueden mostrarse como tabla y como gráfico. Cuando el usuario modifica un valor, tanto la tabla como el gráfico deben actualizarse. La clase `Observable` gestiona la lista de suscriptores y el método `notificar()` que llama a `actualizar()` en cada uno. En el diagrama de secuencia: el observador llama a `establecerEstado()` en el sujeto → el sujeto llama a `notificar()` → cada observador recibe `actualizar()` y consulta el estado con `obtenerEstado()`.

¿Cuándo usarlo?

- Cuando un cambio en un objeto requiere modificar otros sin importar cuántos son.
- Cuando un objeto debe poder notificar a otros sin hacer suposiciones sobre quiénes son.

Participantes

- **Sujeto**: conoce sus observadores; ofrece interfaz para suscribirse y desuscribirse.
- **Observador**: define la interfaz de actualización (`actualizar()`).
- **SujetoConcreto**: almacena el estado de interés y notifica los cambios.
- **ObservadorConcreto**: mantiene referencia al sujeto concreto y sincroniza su estado.

Ventajas e inconvenientes

Ventajas:

- Acoplamiento mínimo entre sujeto y observadores.
- Comunicación mediante difusión (broadcast).
- Permite modificar sujetos y observadores de forma independiente.

Inconvenientes:

- Los observadores no se conocen entre sí ⇒ pueden producirse actualizaciones en cascada inesperadas.
- El protocolo de actualización simple no detalla qué cambió exactamente.

4.2. State (Estado)

State — Idea clave

Permite que un objeto **modifique su comportamiento cuando cambia su estado interno**. Parecerá que el objeto cambia de clase.

Motivación y ejemplo

Una conexión TCP puede estar en estado `TCPEstablecida`, `TCPEscuchando` o `TCPCerrada`. Sin State, los métodos `abrir()`, `cerrar()` y `acuseDeRecibo()` estarían llenos de `switch`/`if` según el estado. Con State, `ConexionTCP` delega en su objeto estado actual: `estado.abrir()`. Cada estado concreto implementa el comportamiento apropiado.

¿Cuándo usarlo?

- Cuando el comportamiento de un objeto depende de su estado y cambia en tiempo de ejecución.
- Cuando existen muchas estructuras condicionales con ramas que dependen del estado.

Participantes

- **Contexto**: mantiene una instancia de EstadoConcreto que define el estado actual.
- **Estado**: declara la interfaz que encapsula el comportamiento de cada estado.
- **EstadoConcreto**: implementa el comportamiento asociado a un estado particular del Contexto.

Ventajas e inconvenientes

Ventajas:

- Concentra todo el comportamiento de un estado en un solo objeto.
- Facilita añadir nuevos estados y transiciones.
- Elimina estructuras condicionales largas.
- Hace explícitas las transiciones entre estados.

Inconvenientes:

- Incrementa el número de clases del sistema.

⋆ Recuerda para el examen

State vs. Strategy: Ambos delegan comportamiento en un objeto. La diferencia es que en **State** el cambio es *interno* (el propio estado puede cambiar el siguiente estado) y los estados se conocen entre sí. En **Strategy** el cliente elige la estrategia *externamente* y las estrategias son independientes entre sí.

4.3. Strategy (Estrategia)

Strategy — Idea clave

Define una **familia de algoritmos**, los encapsula en objetos y los hace **intercambiables**. Permite que el algoritmo varíe con independencia de los clientes que lo usan.

Motivación y ejemplo

Un generador de superficies (`GenSuperficies`) puede interpolar usando IDW, Spline o Kriging. Sin Strategy, `interpolar()` tendría un `switch` largo. Con Strategy, `GenSuperficies` contiene una referencia `-estrategia` a la interfaz `IInterpolador` y delega: `estrategia.calcular()`. El cliente decide qué `EstrategiaConcreta` pasar al contexto.

¿Cuándo usarlo?

- Cuando existen varias clases relacionadas que solo difieren en su comportamiento.
- Cuando se necesitan variantes de un algoritmo.
- Cuando un algoritmo usa datos que los clientes no necesitan conocer.
- Para eliminar condicionales extensas relacionadas con distintos comportamientos.

Participantes

- **Estrategia**: declara la interfaz común a todos los algoritmos.
- **EstrategiaConcreta**: implementa un algoritmo particular.
- **Contexto**: mantiene una referencia a la Estrategia y delega en ella las peticiones del algoritmo.

Ventajas e inconvenientes

Ventajas:

- Define una familia de algoritmos intercambiables.
- Permite modificar cualquier algoritmo sin afectar al contexto.
- Elimina estructuras condicionales.

Inconvenientes:

- Los clientes deben conocer las distintas estrategias disponibles.
- Las estrategias más simples pueden no usar toda la información que reciben.
- Aumenta el número de objetos en el sistema.

4.4. Template Method (Método Plantilla)

Template Method — Idea clave

Define el **esqueleto de un algoritmo en una clase base**, delegando algunos pasos a las subclases. Permite redefinir ciertos pasos sin cambiar la estructura global del algoritmo.

Motivación y ejemplo

La clase abstracta `Aplicacion` tiene el método plantilla `abrirDoc()` que llama en orden a: `sePuedeAbrirDoc()`, `crearDoc()`, `añadirDoc()`, `aPuntoDeAbrirDoc()`, `doc.abrir()`, `doc.leer()`. Algunos de estos métodos son abstractos (deben implementarse en `MiAplicacion`) y otros son

operaciones de enganche con implementación vacía por defecto que las subclases pueden opcionalmente sobreescribir.

¿Cuándo usarlo?

- Para implementar las partes invariantes de un algoritmo dejando las variables a las subclases.
- Para factorizar comportamiento repetido en una clase común (evitar código duplicado).
- Para controlar extensiones de subclases mediante operaciones de enganche.

Participantes

- **ClaseAbstracta**: implementa el método plantilla y declara las operaciones primitivas abstractas y de enganche.
- **ClaseConcreta**: implementa las operaciones primitivas para completar el algoritmo.

Tipos de operaciones invocadas por el método plantilla

- **Primitivas abstractas**: deben ser redefinidas por las subclases (*obligatorio*).
- **Concretas en ClaseAbstracta**: comportamiento común compartido.
- **Operaciones de enganche** (*hooks*): métodos vacíos en el padre que las subclases *pueden* (no deben) redefinir.

Ventajas e inconvenientes

Ventajas:

- Reutilización del código del algoritmo en la clase base.
- Inversión de control: la clase base llama a las subclases ("Hollywood Principle").

Inconvenientes:

- Puede ser difícil de mantener si el algoritmo tiene muchos pasos.
- Las subclases solo pueden variar dentro del esqueleto definido.

⋆ Recuerda para el examen

Template Method vs. Strategy: Template Method usa *herencia* para variar partes de un algoritmo; la variación se define en tiempo de compilación. Strategy usa *composición* para elegir el algoritmo completo; la variación se produce en tiempo de ejecución.

Tablas Resumen para el Examen

5.1. Resumen general de los patrones

Patrón	Categoría	Idea clave	Cuándo usarlo
Abstract Factory	Creación	Interfaz para crear familias de objetos. El cliente solo conoce interfaces.	Familias de productos que deben usarse juntos. Cambio de familia en tiempo de ejecución.
Factory Method	Creación	La subclase decide qué clase instanciar. Un método abstracto que devuelve un Producto.	Clase no puede prever qué clase de objeto crear. Dejar a subclases la elección del producto.
Adapter	Estructural	Convierte una interfaz incompatible en la que espera el cliente.	Reutilizar una clase de terceros cuya interfaz no encaja.
Bridge	Estructural	Separa abstracción de implementación en dos jerarquías independientes.	Evitar explosión de subclases por dos dimensiones de variación.
Composite	Estructural	Árbol de objetos. Hojas y compuestos tienen la misma interfaz.	Jerarquías parte-todo; cliente no distingue hoja de compuesto.
Decorator	Estructural	Añade responsabilidades dinámicamente sin subclasificar.	Extensión flexible de objetos en tiempo de ejecución.
Proxy	Estructural	Sustituto con la misma interfaz para controlar el acceso.	Acceso remoto, lazy loading, control de permisos.
Observer	Comportamiento	Notificación automática 1:N cuando el sujeto cambia.	Mantener consistencia entre objetos dependientes sin acoplarlos.
State	Comportamiento	El objeto delega en un objeto-estado; cambia de estado en tiempo de ejecución.	Comportamiento dependiente del estado; muchos condicionales según estado.
Strategy	Comportamiento	Familia de algoritmos intercambiables encapsulados en objetos.	Variantes de un algoritmo; eliminar condicionales de selección de comportamiento.
Template Method	Comportamiento	Esqueleto de algoritmo en clase base; subclases rellenan los pasos.	Partes fijas y partes variables de un algoritmo; factorizar código común.

5.2. Comparación de patrones similares

Patrones con interfaz idéntica al objeto original

	Proxy	Decorator
Interfaz	Idéntica al sujeto	Idéntica al componente
Propósito	Controlar acceso	Añadir responsabilidades
Quién crea el real	El propio proxy (lazy)	El cliente
Encadenamiento	No (uno a uno)	Sí (capas)

Patrones de creación: Abstract Factory vs. Factory Method

	Abstract Factory	**Factory Method**
Mecanismo	Composición/delegación	Herencia
Qué crea	Familia de productos	Un producto
Quien decide	Objeto fábrica	Subclase
Complejidad	Mayor	Menor
Cambio en ejecución	Sí (cambiar fábrica)	No

Patrones que usan delegación con la misma interfaz

	State	**Strategy**
Quién cambia	El propio estado (interno)	El cliente (externo)
Los objetos se conocen	Sí (transiciones)	No
Propósito	Cambio de comportamiento por estado	Intercambio de algoritmo
Número de impl.	Uno activo a la vez	Uno activo a la vez

Template Method vs. Strategy

	Template Method	**Strategy**
Mecanismo	Herencia	Composición
Granularidad	Pasos del algoritmo	Algoritmo completo
Variación	En compilación	En ejecución
Variante de código	Métodos primitivos	Objeto estrategia

5.3. Composición y patrones que se relacionan

Relaciones entre patrones

- **Composite + Decorator**: ambos usan agregación recursiva. Composite para jerarquías, Decorator para añadir responsabilidades. Se combinan frecuentemente.
- **Abstract Factory + Factory Method**: Abstract Factory suele implementar sus métodos de creación como Factory Methods.
- **Template Method + Factory Method**: el método plantilla puede usar un Factory Method para crear objetos dentro del algoritmo.
- **Observer + Mediator**: Observer distribuye la comunicación (poco acoplado, más reutilizable). Mediator la centraliza (patrón de comunicación más sencillo).
- **Strategy + Bridge**: estructuralmente similares (ambos usan composición con una referencia a una interfaz abstracta), pero Bridge separa abstracción de implementación mientras que Strategy encapsula algoritmos intercambiables.

5.4. Tabla de participantes rápida

Patrón	Participantes (en negrita el más relevante)
Abstract Factory	**FabricaAbstracta**, FabricaConcreta, ProductoAbstracto, ProductoConcreto, Cliente
Factory Method	Producto, ProductoConcreto, **Creador**, **CreadorConcreto**
Adapter	Objetivo, Cliente, **Adaptado**, **Adaptador**
Bridge	**Abstraccion**, AbstraccionRefinada, **Implementador**, ImplementadorConcreto
Composite	**Componente**, **Hoja**, **Compuesto**, Cliente
Decorator	Componente, ComponenteConcreto, **Decorador**, **DecoradorConcreto**
Proxy	Sujeto, **SujetoReal**, **Proxy**
Observer	**Sujeto**, **Observador**, SujetoConcreto, ObservadorConcreto
State	**Contexto**, **Estado**, EstadoConcreto
Strategy	**Estrategia**, EstrategiaConcreta, Contexto
Template Method	**ClaseAbstracta**, ClaseConcreta

⋆ Recuerda para el examen

Preguntas tipo que pueden aparecer en el examen:

1. Dado un diagrama UML, identifica el patrón y nombra los participantes.
2. Explica la diferencia entre los patrones X e Y (frecuente: Decorator/Proxy, State/Strategy, Adapter/Bridge).
3. ¿En qué situación usarías el patrón X? Razona tu respuesta.
4. Indica las ventajas e inconvenientes del patrón X.
5. Dado un problema de diseño, ¿qué patrón(es) aplicarías?

Referencia principal: Gamma, Helm, Johnson, Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
Diapositivas de José Varela Pet, Departamento de Electrónica y Computación, USC, 2025/2026.